I used ChatGPT (GPT-5.6 Sol) on September 26 and 27 to assist with development, testing, review, and documentation. I then compiled, ran, and tested the program myself in Visual Studio Code on my Mac.

1. How AI Was Used

ChatGPT helped interpret the requirements, generate some of the C++ implementation and test driver, suggest boundary and malformed-input tests, review the parser logic, and prepare the README.

2. Exact Prompts Used

Prompt 1:

I am doing an assignment of C++, in which I have to find out only one valid IPv4 address and optionally a port number from a string line. Design the program according to the following prototype of the code: bool extractIPv4(const std::string& str, unsigned long& outAddress, int& outPort); also Important restrictions are: Avoid functions such as atoi, stoi, strtol, sscanf or any other string-to-number parser. Avoid address parsing functions like inet_aton, inet_pton, inet_addr or any others like those. Regular expressions should not be used either. All numeric values should be parsed manually, one character at a time. Also The port must contain 1–5 digits, have a value from 0–65535, and follow the same leading-zero rule. In addition, each of the four parts of an IP address needs to consist of 1-3 digits and lie within the range of 0-255. Leading zeroes are acceptable only when the number is equal to zero. As an additional option, there can be a port number specified in the format :port after the fourth octet. If a colon exists, the entire port must be valid or the entire candidate must fail. A sequence of digits, periods, and colons should be treated as one complete candidate, so the program must not accept a valid substring from inside a malformed candidate. On failure, set outAddress = 0 and outPort = -1. If no port is present, set outPort = -1. Explain first the parsing strategy, then provide a simple C++ implementation that follows these rules

Prompt 2 - Project Setup and Testing

Review the implementation you just generated as if you were grading it strictly. Please Do not assume it is correct as there are always bugs and I need to be able to debug. I am also using Visual Studio Code for all the coding just so you know.

Prompt 3 - Code Review

The current implementation has a partial-match problem. It can parse four valid octets and return success without verifying that the complete candidate token has been consumed.

For example 192.168.1.1. must be rejected because the trailing period is directly adjacent and is part of the same candidate. Please fixx the implementation so that the program first identifies the full maximal sequence containing only digits, periods, and colons, the parser validates that entire sequence, success is allowed only if the parsing position reaches exactly the end of that candidate.

Prompt 4:

The following addresses have to be classified as valid and the following as invalid: 0.0.0.0, 10.0.0.1, 1.2.3.4:0, 1.2.3.4:80, 1.2.3.4:65535 The following addresses have to be considered invalid: 00.0.0.0, 01.2.3.4, 1.2.3.4:01, 1.2.3.4:65536 Also, make sure that any multi-digit number starting with 0 is rejected but the number 0 accepted.

2. Issues found

The first numeric parsing helper checked only the numeric value and digit count. That means an input such as 01 could be converted numerically to 1 and accepted even though the assignment explicitly prohibits leading zeros.

3. AI-Generated vs My Work

Contribution of AI: The first parser structure; suggestions for helper functions to scan tokens and manually parse numbers; the first version of the test driver; malformed and edge cases; and checking of the code for partial match, leading zero, port limit, and candidate token.

My Contribution: The organization of the project in VS Code; the decision on which AI suggestions to take into account; fixing of code or set up errors that were encountered during compilation; running the program manually; adding or modifying test cases; making sure the correct output format is observed; and verifying of 31/31 test cases. The AI gave a first draft and review help, while I had the remaining responsibility for making sure it worked and that code realated to the assignment and that I actually understood it.

4. Modifications and Review

I reviewed the generated code rather than accepting it because it compiled. I verified that the malformed candidate tokens are rejected in full, so 192.168.1.1. is invalid instead of being partially accepted as 192.168.1.1. I also checked leading-zero handling, octet limits of 0-255, port limits of 0-65535, and manual numeric conversion without prohibited library functions.

5. Testing

Main program:

g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o ipv4_parser

I manually tested a normal IPv4 address, an address with port 8080, a malformed address with a trailing period, and the END termination command.

Automated tests:

g++ -std=c++17 -Wall -Wextra -pedantic tests.cpp -o tests

The test case set of 31 cases included proper and embedded addresses, proper and improper port numbers, minimum/maximum limit, leading zeros, missing/extra octets, null entries, additional punctuation marks, multiple colons, and proper address possibilities.

Final result: 31/31 tests passed.

6. Known Limitation / Interpretation

My implementation rejects a line containing two separate valid IPv4 addresses because I interpreted the requirement that exactly one valid address may be extracted per input line literally.

7. Verification Statement

The help of AI was understandable to me including manual digit collection, octet and port validation, leading zero rejection, token validation, parameter output, 32-bit IPv4 generation, and input processing loop. The provided code has been compiled, executed, and tested by me personally throughout the whole process. I can guarantee that it works.